

LAPORAN PRAKTIKUM
POSTTEST 4
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:
Hamam Syamil (2509106073)
Kelas B2'25

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

1. Flowchart

1. Struktur Data & Inisialisasi

Program menggunakan Struct (User, Penyewa, Kost) untuk mengelompokkan data secara teratur dalam Array of Struct. Data penyewa dikelola sebagai nested struct di dalam data kost untuk keterhubungan informasi.

2. Menu Awal (Autentikasi)

Program menyediakan pilihan Register dan Login. Fungsi registerUser bertugas menambah akun dengan validasi kapasitas, sementara loginUser memverifikasi akun dengan batas tiga kali percobaan. Jika login gagal total, program berhenti; jika sukses, pengguna diarahkan ke menu utama.

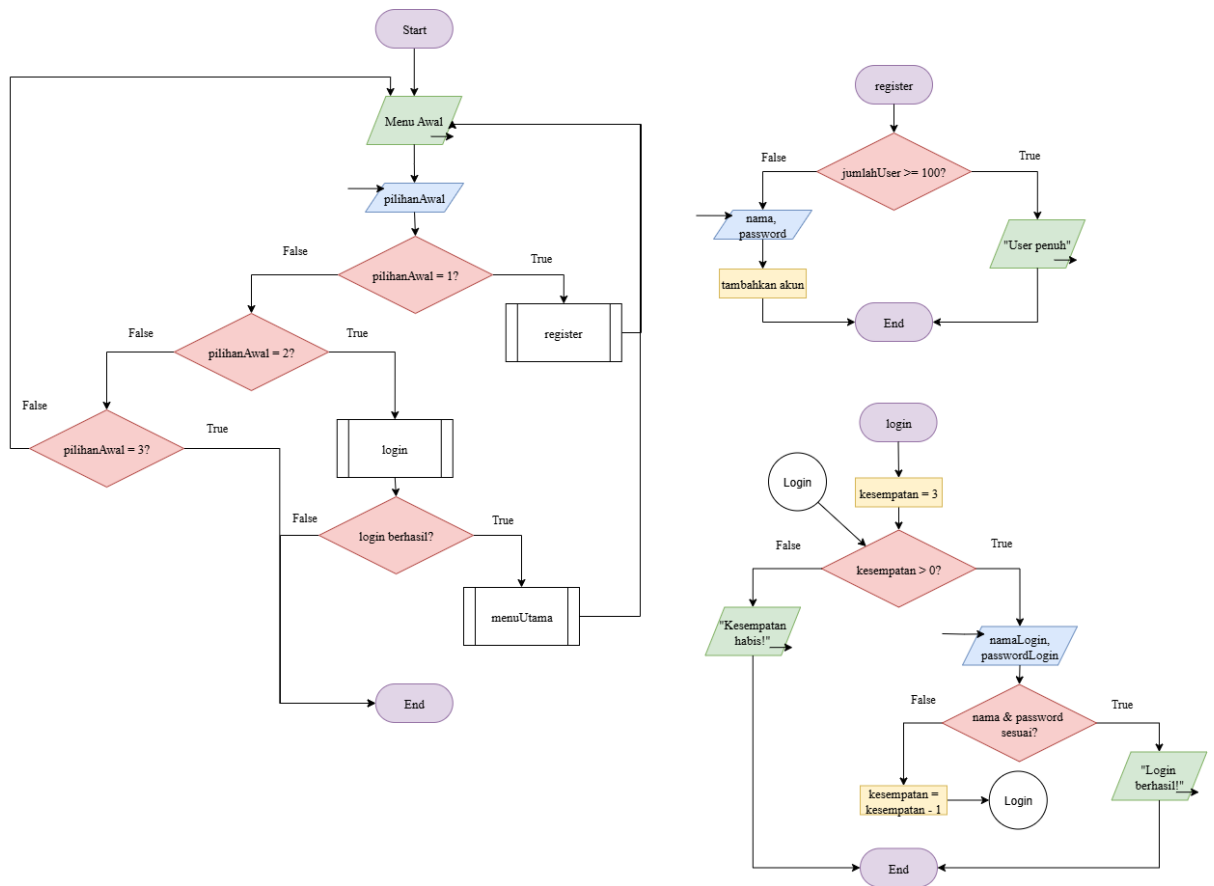
3. Menu Utama (Operasi CRUD)

Menu ini dikelola dalam prosedur menuUtama menggunakan perulangan. Program menyediakan fitur:

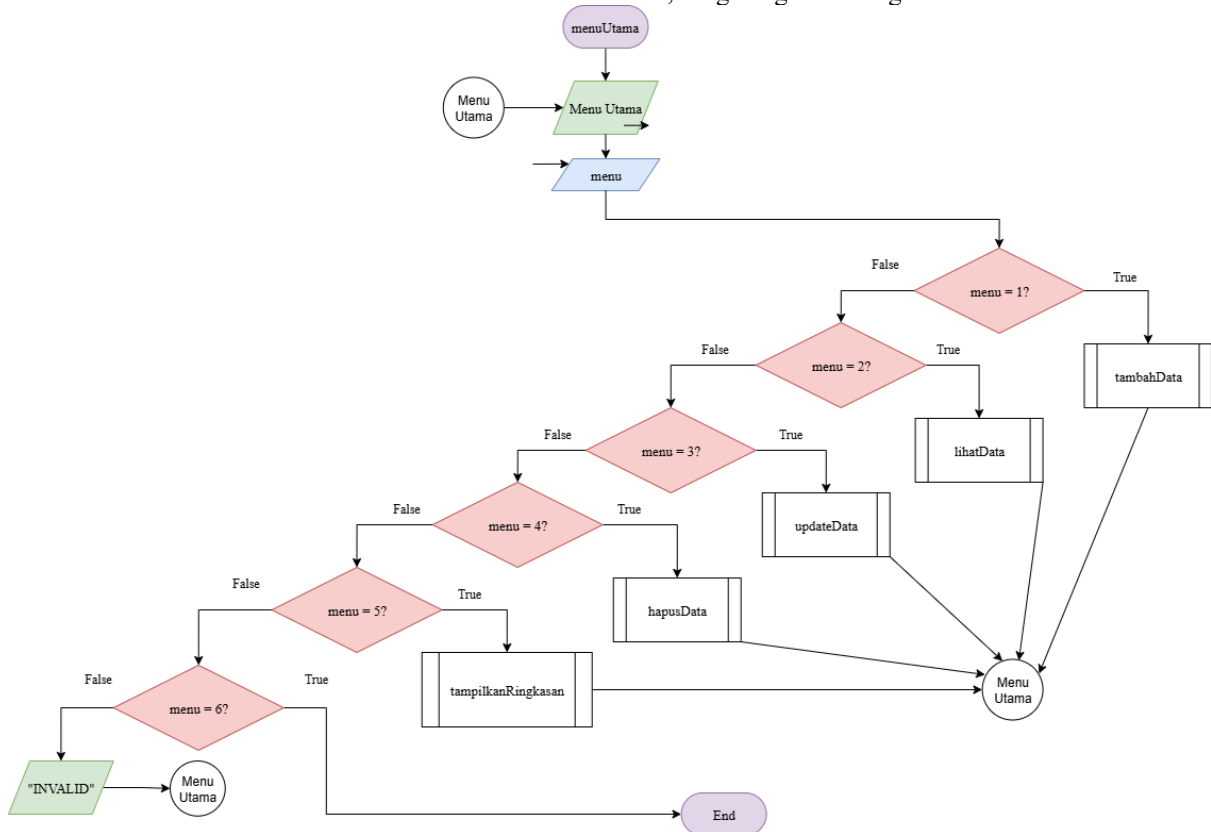
- Tambah & Lihat: Menggunakan fungsi tambahData untuk input dan prosedur lihatData untuk menampilkan tabel.
- Update & Hapus: Keduanya memanfaatkan fungsi rekursif cariKamar untuk menemukan ID yang tepat sebelum data diubah atau dihapus (dengan menggeser indeks array).

4. Fitur Tambahan & Ringkasan

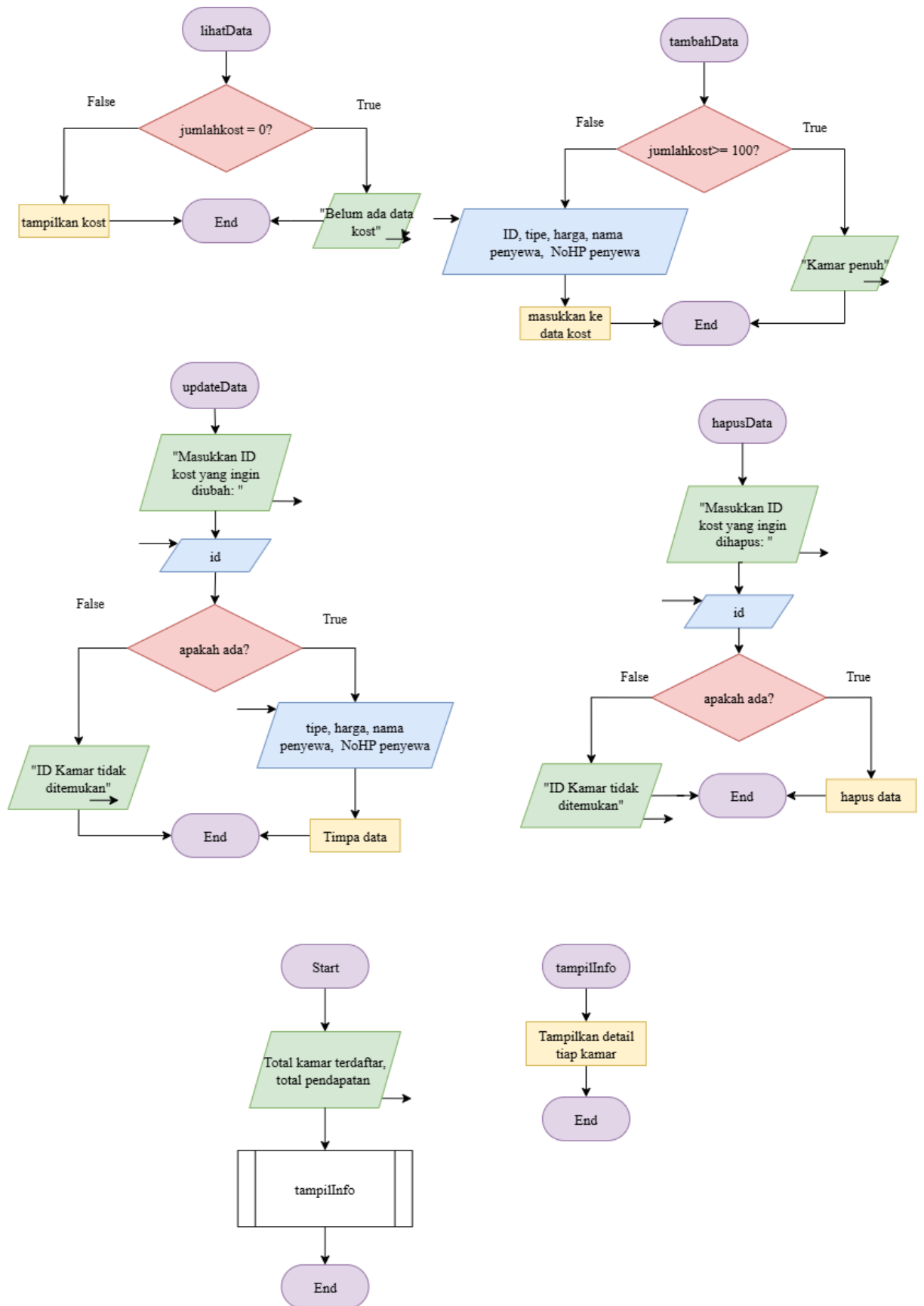
Terdapat fitur Ringkasan yang menggunakan fungsi rekursif totalHarga untuk menghitung pendapatan secara otomatis. Selain itu, program menerapkan Function Overloading pada prosedur tampilInfo, sehingga satu nama fungsi bisa menampilkan detail kamar dalam berbagai format sesuai kebutuhan.



Gambar 1. 1 Flowchart menu awal, fungsi register & login



Gambar 1. 2 Flowchart menu utama



Gambar 1. 3 Flowchart CRUD

2. Deskripsi Singkat Program

Program ini bertujuan untuk menyediakan sistem sederhana dalam mengelola data penyewaan kost secara terstruktur. Dengan adanya fitur login dan register, program memastikan bahwa hanya pengguna yang terdaftar yang dapat mengakses sistem. Selain itu, fitur CRUD membantu pengguna dalam menambah, melihat, mengubah, dan menghapus data kamar serta penyewa dengan lebih mudah. Dengan sistem ini, pengelolaan data kost menjadi lebih praktis, terorganisir, dan mengurangi kesalahan dalam pencatatan data secara manual.

3. Source Code

- PENGATURAN STRUKTUR DATA

Tempat mendefinisikan "wadah" data. User untuk akun, Penyewa untuk profil tamu, dan Kost sebagai data utama yang menggabungkan informasi kamar dan penyewa.

```
struct Penyewa {  
    string nama;  
    string noHP;  
};  
  
struct Kost {  
    int idKamar;  
    string tipe;  
    int harga;  
    Penyewa penyewa;  
};  
struct User {  
    string nama;  
    string password;  
};
```

- FITUR ANTARMUKA

Berisi prosedur tampilHeader untuk menjaga tampilan program tetap konsisten dan rapi di setiap menu.

```
void tampilHeader(string judul) {  
    cout << "\n==== " << judul << " =====\n";  
}
```

- FITUR AUTENTIKASI

Fitur ini bertindak sebagai gerbang masuk program. Terdapat sistem pembatasan percobaan login (maksimal 3 kali). Jika batas terlampaui, program akan melakukan pemutusan sesi secara otomatis untuk mencegah upaya menebak password secara berulang.

```

bool registerUser(User users[], int *jumlahUser) {
    tampilHeader("REGISTER");
    if (*jumlahUser >= 100) {
        cout << "Kapasitas user penuh!\n";
        return false;
    }
    cin.ignore(1000, '\n');
    cout << "Username: ";
    getline(cin, users[*jumlahUser].nama);
    cout << "Password: ";
    getline(cin, users[*jumlahUser].password);

    (*jumlahUser)++;
    cout << "Registrasi berhasil!\n";
    return true;
}

bool loginUser(User users[], int jumlahUser) {
    tampilHeader("LOGIN");
    string namaLogin, passwordLogin;
    int kesempatan = 3;
    cin.ignore(1000, '\n');
    while (kesempatan > 0) {
        cout << "Username      : ";
        getline(cin, namaLogin);
        cout << "Password: ";
        getline(cin, passwordLogin);

        for (int i = 0; i < jumlahUser; i++) {
            if (users[i].nama == namaLogin && users[i].password == passwordLogin) {
                cout << "Login berhasil! Selamat datang, " << namaLogin << "!\n";
                return true;
            }
        }
        kesempatan--;
        if (kesempatan > 0) {
            cout << "Login gagal! Sisa percobaan: " << kesempatan << "\n\n";
        }
    }
    cout << "Kesempatan habis! Program berhenti.\n";
    return false;
}

```

- FITUR PENCARIAN & PERHITUNGAN

Menggunakan logika Rekursif untuk menjamin akurasi data. cariKamar melakukan penelusuran data hingga ke elemen terakhir (base case) untuk memastikan tidak ada data yang terlewat, sementara totalHarga menjamin perhitungan nilai numerik yang presisi tanpa risiko looping yang tak terhingga.

```

int cariKamar(Kost dataKost[], int id, int index, int jumlahKost) {
    if (index >= jumlahKost) return -1;
    if (dataKost[index].idKamar == id) return index;
    return cariKamar(dataKost, id, index + 1, jumlahKost);
}

int totalHarga(Kost dataKost[], int index, int jumlahKost) {
    if (index >= jumlahKost) return 0;
    return dataKost[index].harga + totalHarga(dataKost, index + 1, jumlahKost);
}

```

- **FITUR UTAMA MANAJEMEN DATA**

Fitur utama yang mengelola siklus hidup data kost di dalam sistem:

- Melakukan alokasi data baru ke dalam array dengan memvalidasi batas kapasitas memori yang tersedia.
- Menyajikan data secara terstruktur menggunakan format tabel untuk memudahkan pemantauan seluruh informasi yang tersimpan.
- Melakukan modifikasi pada elemen data spesifik berdasarkan hasil pencarian ID, memastikan informasi tetap aktual tanpa mengubah struktur array.
- Menghapus data terpilih


```

void lihatData(Kost dataKost[], int jumlahKost) {
    tampilHeader("DATA KOST");
    if (jumlahKost == 0) {
        cout << "Belum ada data kost.\n";
        return;
    }
    cout << left
        << setw(5) << "ID"
        << setw(12) << "Tipe"
        << setw(12) << "Harga"
        << setw(20) << "Nama Penyewa"
        << setw(15) << "No HP"
        << endl;
    cout << string(64, '-') << endl;
    for (int i = 0; i < jumlahKost; i++) {
        cout << left
            << setw(5) << dataKost[i].idKamar
            << setw(12) << dataKost[i].tipe
            << setw(12) << dataKost[i].harga
            << setw(20) << dataKost[i].penyewa.nama
            << setw(15) << dataKost[i].penyewa.noHP
            << endl;
    }
}

bool tambahData(Kost dataKost[], int *jumlahKost) {
    tampilHeader("TAMBAH DATA");
    if (*jumlahKost >= 100) {
        cout << "Kapasitas kamar penuh!\n";
        return false;
    }
    dataKost[*jumlahKost].idKamar = *jumlahKost + 1;
    cout << "ID Kamar1: " << dataKost[*jumlahKost].idKamar << endl;
    cin.ignore(1000, '\n');
    cout << "Tipe Kamar : "; getline(cin, dataKost[*jumlahKost].tipe);
    cout << "Harga : "; cin >> dataKost[*jumlahKost].harga;
    cin.ignore(1000, '\n');
    cout << "Nama Penyewa : "; getline(cin, dataKost[*jumlahKost].penyewa.nama);
    cout << "No HP Penyewa : "; getline(cin, dataKost[*jumlahKost].penyewa.noHP);
    (*jumlahKost)++;
    cout << "Data berhasil ditambahkan!\n";
    return true;
}

bool updateData(Kost dataKost[], int jumlahKost) {
    tampilHeader("UPDATE DATA");
    int id;
    cout << "Masukkan ID Kamar yang ingin diubah: ";
    cin >> id;

    int index = cariKamar(dataKost, id, 0, jumlahKost);
    if (index == -1) {
        cout << "ID Kamar tidak ditemukan!\n";
        return false;
    }

    cout << "Tipe Baru: ";
    cin >> dataKost[index].tipe;
    cout << "Harga Baru: ";
    cin >> dataKost[index].harga;
    cout << "Nama Penyewa Baru: ";
    cin >> dataKost[index].penyewa.nama;
    cout << "No HP Baru: ";
    cin >> dataKost[index].penyewa.noHP;
    cout << "Data berhasil diupdate!\n";
    return true;
}

bool hapusData(Kost dataKost[], int *jumlahKost) {
    tampilHeader("HAPUS DATA");
    int id;
    cout << "Masukkan ID Kamar yang ingin dihapus: ";
    cin >> id;

    int index = cariKamar(dataKost, id, 0, *jumlahKost);
    if (index == -1) {
        cout << "ID Kamar tidak ditemukan!\n";
        return false;
    }

    for (int j = index; j < *jumlahKost - 1; j++) {
        dataKost[j] = dataKost[j + 1];
    }
    (*jumlahKost)--;
    cout << "Data berhasil dihapus!\n";
    return true;
}

```

- FITUR INFORMASI & RINGKASAN

Penerapan prosedur dengan nama yang sama namun parameter berbeda. Hal ini memungkinkan sistem menampilkan detail informasi dengan berbagai tingkat kedalaman sesuai kebutuhan bagian program lainnya.

```
void tampilInfo(Kost *k) {
    cout << "[Info Kamar] ID: " << k->idKamar << " | Tipe: " << k->tipe << endl;
}
void tampilInfo(Kost *k, bool tampilHarga) {
    cout << "[Info Kamar] ID: " << k->idKamar
        << " | Tipe: " << k->tipe;
    if (tampilHarga)
        cout << " | Harga: Rp" << k->harga;
    cout << " | Penyewa: " << k->penyewa.nama
        << " | No HP: " << k->penyewa.noHP << endl;
}
void tampilInfo(Kost dataKost[], int index) {
    Kost *ptr = &dataKost[index];
    cout << "[Info Kamar ke-" << index + 1 << "]" << "
        << "ID: " << ptr->idKamar
        << " | Tipe: " << ptr->tipe
        << " | Harga: Rp" << ptr->harga
        << " | Penyewa: " << ptr->penyewa.nama << endl;
}
void tampilRingkasan(Kost dataKost[], int jumlahKost) {
    tampilHeader("RINGKASAN DATA KOST");
    cout << "Total kamar terdaftar : " << jumlahKost << endl;
    cout << "Total pendapatan      : Rp" << totalHarga(dataKost, 0, jumlahKost) << endl;
    cout << "\nDetail setiap kamar:\n";
    for (int i = 0; i < jumlahKost; i++) {
        tampilInfo(dataKost, i);
    }
}
```

- PROGRAM UTAMA

Pusat kendali alur program yang menggunakan perulangan untuk menjaga stabilitas sesi pengguna. Fitur ini memastikan program tidak berhenti sebelum pengguna memberikan instruksi keluar secara eksplisit.

```
void menuUtama(Kost dataKost[], int *jumlahKost) {
    int menu;
    do {
        tampilHeader("MENU UTAMA");
        cout << "1. Tambah Data\n";
        cout << "2. Lihat Data\n";
        cout << "3. Update Data\n";
        cout << "4. Hapus Data\n";
        cout << "5. Ringkasan & Info Kamar\n";
        cout << "6. Keluar\n";
        cout << "Pilihan: ";
        cin >> menu;
        switch (menu) {
            case 1: tambahData(dataKost, jumlahKost);           break;
            case 2: lihatData(dataKost, *jumlahKost);           break;
            case 3: updateData(dataKost, *jumlahKost);           break;
            case 4: hapusData(dataKost, jumlahKost);             break;
            case 5: tampilRingkasan(dataKost, *jumlahKost);       break;
            case 6: cout << "Keluar dari menu utama.\n";         break;
            default: cout << "Pilihan tidak valid!\n";           break;
        }
    } while (menu != 6);
}

int main() {
    User users[100];
    Kost dataKost[100];
    int jumlahUser = 0;
    int jumlahKost = 0;
    int pilihanAwal;
    do {
        tampilHeader("MENU AWAL");
        cout << "1. Register\n";
        cout << "2. Login\n";
        cout << "3. Keluar\n";
        cout << "Pilihan: ";
        cin >> pilihanAwal;
        if (pilihanAwal == 1) {
            registerUser(users, &jumlahUser);
        }
        else if (pilihanAwal == 2) {
            bool berhasil = loginUser(users, jumlahUser);
            if (!berhasil) {
                cout << "Tekan Enter untuk keluar...";
                cin.ignore();
                cin.get();
                return 0;
            }
            menuUtama(dataKost, &jumlahKost);
        }
        else if (pilihanAwal != 3) {
            cout << "Pilihan tidak valid!\n";
        }
    } while (pilihanAwal != 3);
    cout << "\nProgram selesai. Terima kasih!\n";
    cout << "Tekan Enter untuk keluar...";
    cin.ignore();
    cin.get();
    return 0;
}
```

4. Hasil Output

```
===== MENU AWAL =====  
1. Register  
2. Login  
3. Keluar  
Pilihan: 1  
  
===== REGISTER =====  
Username: Hammam Syamil  
Password: 2509106073  
Registrasi berhasil!
```

Gambar 4. 1 Output register

```
===== LOGIN =====  
Username      : Hammam Syamil  
Password: 2509106073  
Login berhasil! Selamat datang, Hammam Syamil!  
  
===== MENU UTAMA =====  
1. Tambah Data  
2. Lihat Data  
3. Update Data  
4. Hapus Data  
5. Ringkasan & Info Kamar  
6. Keluar  
Pilihan: 1
```

Gambar 4. 2 Output login berhasil

```
===== MENU UTAMA =====  
1. Tambah Data  
2. Lihat Data  
3. Update Data  
4. Hapus Data  
5. Ringkasan & Info Kamar  
6. Keluar  
Pilihan: 1  
  
===== TAMBAH DATA =====  
ID Kamar: 1  
Tipe Kamar   : VIP  
Harga        : 2000000  
Nama Penyewa : Syamil  
No HP Penyewa : 081349846151  
Data berhasil ditambahkan!
```

Gambar 4. 3 Output tambah data ke 1

```

===== MENU UTAMA =====
1. Tambah Data
2. Lihat Data
3. Update Data
4. Hapus Data
5. Ringkasan & Info Kamar
6. Keluar
Pilihan: 1

```

```

===== TAMBAH DATA =====
ID Kamar: 2
Tipe Kamar      : Standar
Harga           : 700000
Nama Penyewa    : Hammam
No HP Penyewa   : 151648943180
Data berhasil ditambahkan!

```

Gambar 4. 4 Output tambah data ke 2

```

===== MENU UTAMA =====
1. Tambah Data
2. Lihat Data
3. Update Data
4. Hapus Data
5. Ringkasan & Info Kamar
6. Keluar
Pilihan: 2

```

```

===== DATA KOST =====
ID   Tipe      Harga      Nama Penyewa      No HP
-----
1    VIP        2000000    Syamil            081349846151
2    Standar    700000     Hammam            151648943180

```

Gambar 4. 5 Output lihat data

```

===== UPDATE DATA =====
Masukkan ID Kamar yang ingin diubah: 2
Tipe Baru: Kecil
Harga Baru: 450000
Nama Penyewa Baru: Hammam
No HP Baru: 151648943180
Data berhasil diupdate!

===== MENU UTAMA =====
1. Tambah Data
2. Lihat Data
3. Update Data
4. Hapus Data
5. Ringkasan & Info Kamar
6. Keluar
Pilihan: 2

===== DATA KOST =====
ID   Tipe      Harga      Nama Penyewa      No HP
-----
1    VIP        2000000    Syamil            081349846151
2    Kecil      450000     Hammam            151648943180

```

Gambar 4. 6 Output update data ke 2 lalu lihat data yang telah terupdate

```

===== MENU UTAMA =====
1. Tambah Data
2. Lihat Data
3. Update Data
4. Hapus Data
5. Ringkasan & Info Kamar
6. Keluar
Pilihan: 4

===== HAPUS DATA =====
Masukkan ID Kamar yang ingin dihapus: 2
Data berhasil dihapus!

```

Gambar 4. 7 Output hapus data ke 2

```

===== MENU UTAMA =====
1. Tambah Data
2. Lihat Data
3. Update Data
4. Hapus Data
5. Ringkasan & Info Kamar
6. Keluar
Pilihan: 5

===== RINGKASAN DATA KOST =====
Total kamar terdaftar : 1
Total pendapatan      : Rp20000000

Detail setiap kamar:
[Info Kamar ke-1] ID: 1 | Tipe: VIP | Harga: Rp20000000 | Penyewa: Syamil

```

Gambar 4. 8 Output ringkasan & info kamar

```

===== MENU UTAMA =====
1. Tambah Data
2. Lihat Data
3. Update Data
4. Hapus Data
5. Ringkasan & Info Kamar
6. Keluar
Pilihan: 6
Keluar dari menu utama.

```

Gambar 4. 9 Output keluar dari menu utama

```

===== MENU AWAL =====
1. Register
2. Login
3. Keluar
Pilihan: 2

===== LOGIN =====
Username      : salah
Password: ga benar
Login gagal! Sisa percobaan: 2

Username      : salah
Password: ga benar
Login gagal! Sisa percobaan: 1

Username      : salah
Password: ga benar
Kesempatan habis! Program berhenti.
Tekan Enter untuk keluar...

```

Gambar 4. 10 Output login gagal hingga 3 kali

5. Langkah-langkah GIT

5.1 GIT add

Perintah “git add” digunakan untuk menambahkan file ke staging area sebelum dilakukan commit. Pada gambar yang digunakan “git add .” agar semua perubahan di folder saat ini langsung ditambahkan sekaligus, tanpa harus memilih file satu per satu.

```
● PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl> git add .
```

Gambar 5. 1 git add

5.2 GIT Commit

Perintah “git commit” digunakan untuk menyimpan perubahan dari staging area ke riwayat repository lokal. Pada gambar digunakan opsi -m, dipakai untuk memberi pesan singkat.

```
● PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl> git commit -m ".cpp & .exe"
[main 2c3e90c] .cpp & .exe
 2 files changed, 252 insertions(+)
 create mode 100644 post-test/post-test-apl-4/2509106073-HAMMAMSYAMIL-PT-4.cpp
 create mode 100644 post-test/post-test-apl-4/2509106073-HAMMAMSYAMIL-PT-4.exe
```

Gambar 5. 2 git commit

5.3 GIT Push

Perintah “git push” digunakan untuk mengirim commit dari repository lokal ke repository remote.

```
● PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl> git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 52.59 KiB | 7.51 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Aeloxyl/praktikum-apl
 a07b681..2c3e90c main -> main
```

Gambar 5. 3 git push